

SMART LIBRARY MANAGEMENT SYSTEM

Data Organization and Collection Management using Python

Project Type	Python console-based application
Main Concepts	Strings, Lists, Tuples, Sets, Dictionaries
Purpose	Organize book records and manage common library tasks

Prepared as a beginner-friendly project report

1. Project Overview

The Smart Library Management System is a menu-driven Python program for storing and managing library book records. It lets a user add, display, search, update, borrow, return, and remove books. It also displays available books, removes duplicate category names by using a set, sorts books by title, and produces a small library report.

Important note: This version stores records in memory while the program is running. When the program closes, any changes made during that session are lost. File or database storage can be added as a future improvement.

Project objectives

- Store multiple book records in an organized way.
- Search for a book by title or author.
- Show books that are currently available.
- Find unique categories without duplicates.
- Count total, available, and borrowed books.
- Demonstrate Python strings, lists, tuples, sets, and dictionaries.

2. Data Structure Selection Report

Structure	How it is used	Reason
List	The main library collection contains book records.	Can store many records and can be updated.
Dictionary	Each book stores fields such as ID, title, author, category, price, shelf and availability.	Fields are accessed using clear keys.
Tuple	A sample collection of ISBN strings is kept in a tuple.	Suitable for a fixed sequence of values.
Set	Unique category and author names are collected in sets.	Automatically removes duplicates.
String	Titles, names, IDs and categories are text values; lowercase text is used for searching.	Useful for text storage and matching.

3. Problem Analysis: Eight Core Problems

1. Store multiple books

Use a list named library. Each item in the list is a dictionary representing one book. Add a new dictionary with `append()`.

2. Search by title

Ask for a title keyword, convert it to lowercase, and check whether it appears in each book title.

3. Display available books

Check the available field of every book. Display only records whose value is True.

4. Remove duplicate categories

Build a set from the category field of all book records. A set keeps only unique values.

5. Find author information

Search the author field using a case-insensitive keyword and display matching records.

6. Count available books

Count records for which available is True. Borrowed count is total books minus available books.

7. Organize by genre

Use `sorted()` to arrange the books by category or title. This project includes sorting by title and listing unique categories.

8. Store records in a dictionary

Keep each book's details under descriptive keys, for example title, author, category and available.

4. Algorithm Design

Algorithm A: Add a book

1. Start.
2. Input book ID.
3. If ID already exists, show an error and stop this operation.
4. Input title, author, category, shelf and price.
5. Validate that text fields are not blank and price is a non-negative number.
6. Create a dictionary for the new book and set available to True.
7. Append the dictionary to the library list.
8. Display success message.

Algorithm B: Search by title

1. Start.
2. Input a title keyword.
3. Convert the keyword and each title to lowercase.
4. Check each title for the keyword.
5. Display matching books; if none match, display no books found.
6. Stop.

Algorithm C: Borrow a book

1. Start.
2. Input book ID and find the matching record.
3. If no record exists, display book not found.
4. If the book is already borrowed, display a message.
5. Otherwise set available to False and confirm the borrow.
6. Stop.

Algorithm D: Count available books

1. Start.
2. Set available_count to zero.
3. Visit each book record.
4. If available is True, increase available_count by one.
5. Calculate borrowed_count as total minus available_count.
6. Display the counts.
7. Stop.

Algorithm E: Show unique categories

1. Start.
2. Read category from every book.
3. Put category values into a set.
4. Sort the unique category names for neat display.
5. Display categories.
6. Stop.

Algorithm F: Search by author

1. Start.
2. Input author name or keyword.
3. Compare the lowercase keyword with each lowercase author name.
4. Display matching book records.

5. If no record matches, display no books found.
6. Stop.

Algorithm G: Sort books

1. Start.
2. Take the library list.
3. Sort records using each book's lowercase title as the key.
4. Display the sorted records.
5. Stop.

Algorithm H: Update a book

1. Start.
2. Input book ID and search for the record.
3. If it is not found, display an error.
4. Ask which field to update.
5. Read and validate the new value.
6. Change the selected dictionary field.
7. Display success message.
8. Stop.

5. Flowcharts

The following flowcharts show the main decision steps. They are text-based flowcharts so they remain readable in the PDF.

Flowchart 1: Add a Book

```
graph TD; START([START]) --> EnterID[Enter book ID]; EnterID --> IDExists{ID already exists?}; IDExists -- Yes --> ShowDup[Show duplicate ID message] --> END1([END]); IDExists -- No --> EnterDetails[Enter book details and price]; EnterDetails --> DetailsValid{Details valid?}; DetailsValid -- No --> ShowError[Show error] --> END2([END]); DetailsValid -- Yes --> CreateDict[Create dictionary and append to list]; CreateDict --> ShowSuccess[Show success]; ShowSuccess --> END3([END]);
```

(START)

↓

[Enter book ID]

↓

■ ID already exists? ■

Yes → [Show duplicate ID message] → (END)

No ↓

[Enter book details and price]

↓

■ Details valid? ■

No → [Show error] → (END)

Yes ↓

[Create dictionary and append to list]

↓

[Show success]

↓

(END)

Flowchart 2: Search by Title

(START)

↓

[Enter keyword]

↓

[Check each book title]

↓

■ Match found? ■

Yes → [Display matching books]

No → [Display 'No books found']

↓

(END)

Flowchart 3: Borrow a Book

(START)

↓

[Enter book ID]

↓

■ Book found? ■

No → [Show 'Book not found'] → (END)

Yes ↓

■ Book available? ■

No → [Show 'Already borrowed'] → (END)

Yes ↓

[Set available = False]

↓

[Show success]

↓

(END)

Flowchart 4: Count Available Books

(START)

↓

[Set count = 0]

↓

[Visit each book]

↓

■ available is True? ■

Yes → [count = count + 1]

No → [Continue to next book]

↓

■ More books? ■

Yes → [Visit next book]

No ↓

[Display available count]

↓

(END)

6. Pseudocode

Add book

```
BEGIN
  INPUT book_id
  IF book_id already exists THEN
    DISPLAY "Duplicate ID"
  ELSE
    INPUT title, author, category, shelf, price
    IF details are valid THEN
      CREATE book dictionary
      SET available = TRUE
      APPEND book to library
      DISPLAY "Book added"
    ELSE
      DISPLAY "Invalid details"
    ENDIF
  ENDIF
ENDIF
END
```

Search by title

```
BEGIN
  INPUT keyword
  SET found = FALSE
  FOR each book in library
    IF lowercase(keyword) is in lowercase(book.title) THEN
      DISPLAY book
      SET found = TRUE
    ENDIF
  ENDFOR
  IF found = FALSE THEN DISPLAY "No books found"
END
```

Update book

```
BEGIN
  INPUT book_id
  FIND book by ID
  IF book is not found THEN
    DISPLAY "Book not found"
  ELSE
    INPUT field to update and new value
    VALIDATE new value
    UPDATE selected dictionary field
    DISPLAY "Updated"
  ENDIF
END
```

Unique categories

```
BEGIN
  SET categories = empty set
  FOR each book in library
    ADD book.category to categories
  ENDFOR
  DISPLAY sorted(categories)
END
```

Count available books

```
BEGIN
  SET count = 0
  FOR each book in library
    IF book.available = TRUE THEN
      count = count + 1
    ENDIF
  ENDFOR
  DISPLAY count
END
```

Search author

```
BEGIN
  INPUT author_keyword
  FOR each book in library
    IF lowercase(author_keyword) is in lowercase(book.author) THEN
      DISPLAY book
    ENDIF
  ENDFOR
END
```

Sort by title

```
BEGIN
  SORT library by lowercase(book.title)
  DISPLAY sorted library
END
```

Borrow or return

```
BEGIN
  INPUT book_id
  FIND book by ID
  IF book not found THEN DISPLAY "Book not found"
  ELSE IF borrowing AND book.available = FALSE THEN
    DISPLAY "Already borrowed"
  ELSE IF borrowing THEN
    SET book.available = FALSE
    DISPLAY "Borrowed successfully"
  ELSE IF book.available = TRUE THEN
    DISPLAY "Already available"
  ELSE
    SET book.available = TRUE
    DISPLAY "Returned successfully"
  ENDIF
END
```

7. Complete Python Program

Save the following code as **library_management.py**. Run it with Python 3. The program includes sample books so it can be tested immediately.

```
# Smart Library Management System
# Beginner-friendly menu-driven Python project

library = [
    {
        "book_id": "B001",
        "title": "Python Basics",
        "author": "A. Sharma",
        "category": "Programming",
        "price": 350.0,
        "shelf": "A1",
        "available": True
    },
    {
        "book_id": "B002",
        "title": "Computer Fundamentals",
        "author": "R. Patil",
        "category": "Computer Science",
        "price": 280.0,
        "shelf": "A2",
        "available": True
    },
    {
        "book_id": "B003",
        "title": "Learning Python",
        "author": "A. Sharma",
        "category": "Programming",
        "price": 420.0,
        "shelf": "A1",
        "available": False
    }
]

# A tuple is used for fixed book identifiers in this demonstration.
ISBN_EXAMPLES = ("978-0000000001", "978-0000000002", "978-0000000003")

def read_non_empty(prompt):
    """Read a value and reject blank input."""
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("Input cannot be empty. Please try again.")

def find_book(book_id):
    """Return the book dictionary for an ID, or None if not found."""
    for book in library:
        if book["book_id"].lower() == book_id.lower():
            return book
    return None

def add_book():
    print("\n--- Add Book ---")
    book_id = read_non_empty("Enter book ID: ")
    if find_book(book_id):
        print("A book with this ID already exists.")
        return

    title = read_non_empty("Enter title: ")
    author = read_non_empty("Enter author: ")
    category = read_non_empty("Enter category/genre: ")
    shelf = read_non_empty("Enter shelf location: ")

    try:
        price = float(input("Enter price: "))
```

Complete Python Program (continued)

```
        if price < 0:
            print("Price cannot be negative.")
            return
    except ValueError:
        print("Please enter a valid number for price.")
        return

    new_book = {
        "book_id": book_id,
        "title": title,
        "author": author,
        "category": category,
        "price": price,
        "shelf": shelf,
        "available": True
    }
    library.append(new_book)
    print("Book added successfully.")

def display_books(books=None):
    if books is None:
        books = library
    if not books:
        print("No books found.")
        return

    print("\nID | Title | Author | Category | Price | Shelf | Status")
    print("-" * 78)
    for book in books:
        status = "Available" if book["available"] else "Borrowed"
        print(
            f'{book["book_id"]} | {book["title"]} | {book["author"]} | '
            f'{book["category"]} | {book["price"]:.2f} | '
            f'{book["shelf"]} | {status}'
        )

def search_title():
    keyword = read_non_empty("Enter title or keyword to search: ").lower()
    matches = [
        book for book in library
        if keyword in book["title"].lower()
    ]
    display_books(matches)

def search_author():
    author_name = read_non_empty("Enter author name or keyword: ").lower()
    matches = [
        book for book in library
        if author_name in book["author"].lower()
    ]
    display_books(matches)

def update_book():
    book_id = read_non_empty("Enter book ID to update: ")
    book = find_book(book_id)
    if not book:
        print("Book not found.")
        return

    print("1. Title 2. Author 3. Category 4. Price 5. Shelf")
    choice = input("Choose field to update: ").strip()
    fields = {
        "1": "title",
        "2": "author",
```

Complete Python Program (continued)

```
        "3": "category",
        "4": "price",
        "5": "shelf"
    }
    if choice not in fields:
        print("Invalid choice.")
        return

    field = fields[choice]
    if field == "price":
        try:
            new_value = float(input("Enter new price: "))
            if new_value < 0:
                print("Price cannot be negative.")
                return
        except ValueError:
            print("Invalid price.")
            return
    else:
        new_value = read_non_empty(f"Enter new {field}: ")

    book[field] = new_value
    print("Book details updated successfully.")

def borrow_book():
    book_id = read_non_empty("Enter book ID to borrow: ")
    book = find_book(book_id)
    if not book:
        print("Book not found.")
    elif not book["available"]:
        print("This book is already borrowed.")
    else:
        book["available"] = False
        print("Book borrowed successfully.")

def return_book():
    book_id = read_non_empty("Enter book ID to return: ")
    book = find_book(book_id)
    if not book:
        print("Book not found.")
    elif book["available"]:
        print("This book is already marked as available.")
    else:
        book["available"] = True
        print("Book returned successfully.")

def remove_book():
    book_id = read_non_empty("Enter book ID to remove: ")
    book = find_book(book_id)
    if not book:
        print("Book not found.")
        return
    library.remove(book)
    print("Book removed successfully.")

def show_available():
    available_books = [book for book in library if book["available"]]
    display_books(available_books)

def show_categories():
    # A set removes duplicate category names.
    categories = {book["category"] for book in library}
    print("Unique categories:")
```

Complete Python Program (continued)

```
for category in sorted(categories):
    print("-", category)

def sort_books():
    sorted_books = sorted(library, key=lambda book: book["title"].lower())
    display_books(sorted_books)

def library_report():
    total = len(library)
    available_count = sum(1 for book in library if book["available"])
    borrowed_count = total - available_count
    categories = {book["category"] for book in library}
    authors = {book["author"] for book in library}
    total_value = sum(book["price"] for book in library)

    print("\n--- Library Report ---")
    print("Total books:", total)
    print("Available books:", available_count)
    print("Borrowed books:", borrowed_count)
    print("Unique categories:", len(categories))
    print("Unique authors:", len(authors))
    print(f"Total book value: {total_value:.2f}")

def demonstrate_collections():
    print("\n--- Collection Demonstration ---")

    # String: title text can be searched and converted to lowercase.
    sample_title = "Python Basics"
    print("String in lowercase:", sample_title.lower())
    print("Does title contain 'Python'?", "Python" in sample_title)

    # List: the library is a list of book records.
    print("Number of books in list:", len(library))

    # Tuple: fixed values cannot be changed item-by-item.
    print("ISBN tuple:", ISBN_EXAMPLES)

    # Set: only unique values are retained.
    category_set = {book["category"] for book in library}
    print("Unique categories:", category_set)

    # Dictionary: a book record stores named fields.
    if library:
        print("First book title from dictionary:", library[0]["title"])

def show_menu():
    print("\n===== SMART LIBRARY MANAGEMENT SYSTEM =====")
    print("1. Add a book")
    print("2. Display all books")
    print("3. Search by title")
    print("4. Search by author")
    print("5. Update book details")
    print("6. Borrow a book")
    print("7. Return a book")
    print("8. Remove a book")
    print("9. Show available books")
    print("10. Show unique categories")
    print("11. Sort books by title")
    print("12. Library report")
    print("13. Demonstrate string/list/tuple/set/dictionary")
    print("0. Exit")

def main():
```

Complete Python Program (continued)

```
while True:
    show_menu()
    choice = input("Enter your choice: ").strip()

    if choice == "1":
        add_book()
    elif choice == "2":
        display_books()
    elif choice == "3":
        search_title()
    elif choice == "4":
        search_author()
    elif choice == "5":
        update_book()
    elif choice == "6":
        borrow_book()
    elif choice == "7":
        return_book()
    elif choice == "8":
        remove_book()
    elif choice == "9":
        show_available()
    elif choice == "10":
        show_categories()
    elif choice == "11":
        sort_books()
    elif choice == "12":
        library_report()
    elif choice == "13":
        demonstrate_collections()
    elif choice == "0":
        print("Thank you for using the library system.")
        break
    else:
        print("Invalid choice. Please select a menu number.")

if __name__ == "__main__":
    main()
```


8. Program Testing Report

Testing checks whether the main functions behave as expected. The expected outputs below are based on the sample data and the program logic.

Test ID	Test / Input	Expected result
T1	Display all books	The three sample book records are displayed.
T2	Search title: Python	Python Basics and Learning Python are displayed.
T3	Search title: History	No books found.
T4	Search author: A. Sharma	Two books by A. Sharma are displayed.
T5	Show available books	B001 and B002 are displayed; B003 is borrowed.
T6	Show unique categories	Computer Science and Programming appear once each.
T7	Borrow B001	B001 becomes unavailable and success is displayed.
T8	Borrow B003	Message says the book is already borrowed.
T9	Return B003	B003 becomes available.
T10	Add an existing ID B001	Duplicate ID message appears; no duplicate is added.
T11	Enter invalid price	Program displays a validation message.
T12	Enter menu choice 0	Program exits with a thank-you message.

Sample Trace Table: Borrowing B001

Step	Condition / Action	State of B001
1	Initial sample data	available = True
2	User selects Borrow and enters B001	Book is found
3	Check availability	True, so borrowing is allowed
4	Update record	available = False
5	Show result	Book borrowed successfully

9. README / How to Run

Project name: Smart Library Management System

Language: Python 3

Requirements: Python 3 installed; no external libraries are required.

Steps:

1. Open a text editor or Python IDE.
2. Copy the complete program from Section 7.
3. Save the file as `library_management.py`.
4. Open a terminal in the folder where the file is saved.
5. Run `python library_management.py` (on some systems use `python3 library_management.py`).
6. Choose a number from the menu and follow the prompts.

Limitations: Records are stored in memory only. This is a single-user console program and it does not include login, due dates, fines, or permanent database storage.

Future improvements: Save records to a JSON/CSV file or database, add user login, track borrowers and due dates, and create a graphical user interface.

10. Conclusion

The Smart Library Management System demonstrates how Python collection types can be combined to solve a practical data-management problem. Lists store multiple books, dictionaries store each book's details, tuples represent fixed sample values, sets provide unique categories, and strings support text searching. The menu-driven program covers common library operations and includes test cases for checking its behavior.

Submission checklist

- Project report PDF
- Python source file: library_management.py
- README instructions
- Test report and trace table
- Flowcharts and pseudocode
- Git repository with clear commit messages (if required by your assignment)